

Titan — Antigravity Build Brief

Paste everything from "## 1. Mission" down into Antigravity as your first task. Read the "Before you send this" notes above it first — those are for you, not Antigravity.

Before you send this (read first)

1. **The background video is still a placeholder you don't own.** It's a personal asset on someone else's account. It's kept below only so the layout has something to render against — swap it for a video you've generated or licensed yourself before this goes anywhere public. This is the one decision I can't make for you.
 2. **Fonts, contact details and auth method are now decided** (Sections 3, 5, 7) so Antigravity has nothing left to guess at.
 3. **The chat box is wired to real fields, not the fake model list.** The UI's model-picker slot now selects a real **collection**; the effort-picker slot now selects Titan's real **mode** (auto/single/agentive). Voice input, image attachments and speech-to-text are removed — Titan doesn't support them, and faking them would violate the no-mock-data rule.
 4. **This is new scope** relative to the WBS (which budgeted a plain Streamlit dashboard, not a marketing site). Add a line for "Homepage + Login + Auth backend" to workstream 9.0 once this exists.
 5. **Add the master documentation to the project** (the Word doc and the WBS) so Antigravity can read the full API contract, data model and architecture directly — tell it explicitly to treat `(docs/)` as authoritative over anything it infers from the code.
-

1. Mission

Build **Titan**, a real, working web application — not a prototype, not a mockup, not a demo with sample data. Every screen must be backed by the actual Titan API described in Section 6. If the backend is unreachable, the UI must show a clear error state — it must never substitute placeholder, sample, or invented content to "look done."

Non-negotiable rules, in priority order:

1. No mock data, no hardcoded sample answers, no fake citations, no simulated streaming. Every response the user sees comes from a real API call.
 2. No invented AI model names, no fake user accounts, no seeded "demo" documents unless I explicitly upload them myself through the real ingestion endpoint.
 3. If an endpoint isn't built yet, the UI must show "Not available yet" — never a fabricated success state.
 4. Loading states are real (waiting on a real request), not fixed-duration fake spinners.
 5. Ask me before inventing any copy, statistic, or feature not described in this brief.
-

2. Pages to build

Page	Route	Purpose
Homepage	<code>/</code>	Public marketing page (Section 3)
Login	<code>/login</code>	Real authentication against the API (Section 6.4)
Sign up	<code>/signup</code>	Real account creation, if the backend supports it; otherwise show "Contact your admin" — do not fake registration
App / Ask	<code>/app</code>	The authenticated chat interface (Section 4), talking to <code>/v1/ask</code> and <code>/v1/ask/stream</code>
Documents	<code>/app/documents</code>	Lists real ingested documents via <code>GET /v1/documents</code> (can be minimal for v1)

Do not build additional pages (pricing, blog, about) unless I ask. Do not invent nav destinations that lead nowhere.

3. Homepage

Full-bleed cinematic background + centered hero + the Titan Ask box centered on the page, exactly as described below. One composition, dark theme.

Brand and metadata

- Brand: **Titan**
- Document title: `Ask Your Documents, Not a Search Bar`
- Theme color: `■ #0B1420` (deep navy-black, matches the product's navy identity — see brand notes below)
- `color-scheme: dark`

Background video

```
html<video class="background" autoplay muted loop playsinline disablepictureinpicture<source src="https://d8j0ntlcm91z4.cloudfront.net/user_38xzZboKViGWJ0ttwIXH07</video>
```

Mark this clearly in code comments as **TEMPORARY_ASSET — replace before production**. CSS as in the original spec (`position: absolute; inset: 0; z-index: -3; object-fit: cover;` etc.), plus the same vignette overlay.

Fonts

- **Body / UI:** Inter or, if you want something less common, "Public Sans" — variable weight 100–900, `(:root { font-family: "Inter", Arial, sans-serif; })`
- **Headline / display:** "Fraunces" or "General Sans" at weight 500 — used only for `(.hero-title)`, same treatment as the original `((letter-spacing: -2.1px)`, subtle stroke, `(text-shadow)`.

Copy (Titan, not Vantage)

Headline (two lines):

1. `(Stop Searching.)` — white, `(.line-one)`
2. `(Start Asking.)` — `(rgba(211,207,207,.78))`, `(.line-two)`

Body copy, under the headline:

Your knowledge is scattered across a hundred documents.

Titan reads them all, finds the answer, and shows you

exactly where it came from — every time.

Nav: `(Home)` (active), `(How It Works)`, `(Security)`, `(Contact)`

Time panel: keep the glass time panel from the reference, but relabel it something Titan-relevant — e.g. label `(Status)`, value `(All systems operational)` (this must reflect a **real** health check against `(GET /v1/health)`, not a static string — poll it every 30s, and show `(Degraded)` or `(Unavailable)` truthfully if the check fails).

Header CTA: `(Sign Up)` → `(/signup)` **Primary hero CTA:** `(Try Titan)` → scrolls to / focuses the centered Ask box (Section 4), not a dead link **Demo card copy, if you keep the small glass "watch" card from the reference:** `(See It Work)` — must link to a real recorded demo video you provide, or be removed entirely rather than pointing at nothing

Centered Ask box

Directly below the headline/body, **centered horizontally in the viewport** (not bottom-left like the reference), sits the Titan Ask box — see Section 4 for its full spec. This is the visual and functional centerpiece of the homepage.

New: sections below the fold (extends the original single-viewport layout)

The reference spec was a no-scroll, single-viewport hero only. For Titan, allow scrolling below the hero and add:

1. **"How Titan Works"** — three short steps, plain content, no invented statistics: *Ask a question* → *Titan searches your documents* → *Get a cited, verified answer*. Each step gets a short one-line description and a simple icon (no stock photography).

2. **"Built for trust"** — three short points pulled directly from the master documentation's real design (do not invent numbers): grounded answers with citations, verified before you see them, full traceability of every request. No fabricated percentages or customer logos.
3. **Footer** — standard site footer:
 - Left: Titan logo + one-line tagline
 - Middle: link columns — **Product** (How It Works, Security), **Company** (Contact), **Legal** (Privacy, Terms) — link only pages that actually exist; if a page doesn't exist yet, omit the link rather than pointing at a 404
 - Right: contact — email (`srujeethvisinigiri@gmail.com`), phone (`+91 91368 92409`)
 - Bottom bar: (`© 2026 Titan. All rights reserved.`)

Do not add pricing, testimonials, investor logos, or app-store badges — none of that exists for this project.

Login and Sign up pages

Plain, centered forms on the same dark background (no video needed here — static background is fine). Email + password fields, real submission to the auth endpoint (Section 6.4). Show real validation errors returned by the API. No "Sign in with Google" or other providers unless the backend actually supports them.

4. The Titan Ask box (centered chat component)

Keep the **visual design and micro-interactions** of the reference (`PromptInput`) component (the expand/collapse animation, the spring easing, the attachment tray, the pill-shaped send button, the model-select dropdown pattern) — it's a well-built piece of UI. But change what it *does*:

Remove

- Voice recording / microphone / speech-to-text — not part of Titan.
- Image attachment upload — Titan's ingestion is an admin flow (`POST /v1/ingest`), not something end users drag into a chat box. Remove the paperclip/attachment tray entirely for v1.
- The fake model list (`GPT 5.5`, `Opus 4.8`, etc.) and its icons.

Replace with real controls

- **Left pill (was "model select")**: becomes **collection select** — populated from (`GET /v1/documents`) or a real (`GET /v1/collections`) endpoint if one exists; if not, hardcode the single collection name Titan is actually serving, clearly sourced from config, not invented.

- **Second pill (was "effort" Low/Medium/Max):** becomes **mode:** `Auto`, `Single`, `Agentic` — mapped directly to the `mode` field in the real `POST /v1/ask` request body.
- **Send button:** submits to `POST /v1/ask/stream` (Section 6.3). While streaming, show the real agent steps as they arrive (`step_started`/`step_completed` events) in place of any fake "thinking" animation.

Rendering the response

- Stream tokens into the answer area as `token` events arrive.
 - Render citations as clickable chips using the real `citations[]` array from the response; clicking one should show the real chunk source and section (from the API), not placeholder text.
 - Show the real confidence breakdown (`retrieval`, `citation_coverage`, `completeness`) from the response, using the actual numbers returned.
 - If the API returns `status: "abstained"`, render the real abstention card (`found`, `missing`, `suggested_documents`) — do not paper over it with a generic "I don't know."
 - If the request fails (network error, 5xx, timeout), show a real error state with the actual error code and `trace_id` from the response body (Section 9.11 error catalogue) — never swallow the error into a fake answer.
-

5. Authentication (build this — it does not exist yet)

The backend has no auth today. Build the hardened version below — this is the actual security bar, not a placeholder to weaken later.

Password storage

- Hash with **Argon2id** (`argon2-cffi`), not bcrypt. Never store or log plaintext passwords.
- Minimum password length 12 characters. No forced complexity rules (no "must contain a symbol") — length matters more than character-class rules.

Tokens

- **Access token:** JWT, HS256, 15-minute expiry, signed with a secret from an environment variable (never hardcoded).
- **Refresh token:** opaque random token, 7–30 day expiry, stored hashed in a `refresh_tokens` table in Postgres.
- **Rotation:** every refresh issues a new refresh token and immediately invalidates the old one. Track revoked/used tokens in Redis so a reused (stolen) refresh token is rejected and the whole session family is revoked.

- **Delivery:** set both tokens as `(httpOnly; Secure; SameSite=Strict)` cookies. Do not store either token in `localStorage` or `sessionStorage` — that's readable by any injected script.
- **Logout** must actually revoke the refresh token server-side, not just clear the cookie client-side.

Abuse prevention

- Rate limit `(/v1/auth/login)`: lock an email + IP combination out for 15 minutes after 5 failed attempts in 10 minutes (Redis-backed, e.g. `(slowapi)`).
- Lock `(CORS)` to the real deployed frontend origin only — no wildcard `(*)`.
- Log every login attempt (success and failure) with timestamp, email, and IP to the audit log described in the master documentation's security section — this is a real, not cosmetic, requirement.

Backend (FastAPI), endpoints:

- `(POST /v1/auth/signup)` — `{email, password}`; Argon2id-hash before storing; reject duplicate emails with a real 409.
- `(POST /v1/auth/login)` — `{email, password}`; on success, set the access + refresh cookies described above; on failure, a real 401, subject to the rate limit.
- `(POST /v1/auth/refresh)` — reads the refresh cookie, rotates it, issues a new access token.
- `(POST /v1/auth/logout)` — revokes the refresh token server-side, clears both cookies.
- `(GET /v1/auth/me)` — requires a valid access token; returns `{id, email}`; real 401 if missing/expired/invalid.
- Protect `(/v1/ask)`, `(/v1/ask/stream)`, `(/v1/documents)` behind the same check.

Optional stretch, only if there's time (do not let this block the rest of the build):

- TOTP two-factor authentication (`(pyotp)`), opt-in per user, with a `(POST /v1/auth/2fa/enable)` and a verification step added to login.

Frontend:

- `(/signup)` and `(/login)` submit to the real endpoints above and show the real validation and rate-limit errors the API returns.
- The browser never touches the raw tokens — they live only in `httpOnly` cookies the browser sends automatically.
- On a `(401)` from any `(/v1/*)` call, first try `(/v1/auth/refresh)` once; if that also fails, redirect to `(/login)`. Never render the app in a half-authenticated fake state.

Deployment (note, not Antigravity's job to configure, but must not be undermined by app code): this only holds up over real HTTPS with HSTS enabled. Don't build anything that assumes plain HTTP in production.

6. Docker (the whole app, not just the infrastructure)

The existing `(docker-compose.yml)` only runs backing infrastructure (Postgres, Redis, Qdrant, MinIO, Jaeger) for local development, where the API itself runs natively via `(uvicorn)`. For this build, extend it so the **entire application** — API and frontend included — runs from one command.

Add two Dockerfiles:

- `(backend/Dockerfile)` — multi-stage build: install dependencies, copy `(src/)`, run with `(uvicorn titan.api.main:app --host 0.0.0.0 --port 8000)` (no `(--reload)` in the container image; reload is a dev-only concern, not a container concern).
- `(frontend/Dockerfile)` — multi-stage build: install, build the production bundle, serve it (e.g. `(next start)` or a static file server), listening on `(0.0.0.0)`.

Extend `(docker-compose.yml)` with two new services, alongside the existing `(postgres)`, `(redis)`, `(qdrant)`, `(minio)`, `(jaeger)`:

yaml

```
api:
  build: ./backend
  env_file: .env
  ports: ["8000:8000"]
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_started }
    qdrant: { condition: service_started }

frontend:
  build: ./frontend
  env_file: .env
  environment:
    - NEXT_PUBLIC_API_URL=http://localhost:8000
  ports: ["3000:3000"]
  depends_on: [api]
```

- Services talk to each other by **service name**, not `(localhost)` (e.g. the API's `(DATABASE_URL)` is `(postgresql+asyncpg://titan:titan@postgres:5432/titan)`, not `(@localhost:5432)`). `(localhost)` only applies from the host machine's browser hitting published ports.
- Add real `(healthcheck:)` blocks to `(postgres)` (already has one), `(api)` (`(curl -f http://localhost:8000/v1/health)`), and `(frontend)`, and use `(depends_on:)`

`condition: service_healthy`) so services don't start against a database that isn't ready yet.

- One `.env` at the project root, read by both `docker-compose.yml` and the app containers via `env_file` — no secrets hardcoded into any Dockerfile or compose service.
- **One command must bring up the entire working application:** `docker compose up --build` → homepage reachable at `http://localhost:3000`, API at `http://localhost:8000/docs`. This is the actual acceptance test for "it's done" — if that command doesn't produce a working app end to end, the build isn't finished.
- Keep local hot-reload development working too: developers can still run the API with `uvicorn --reload` and the frontend dev server natively against the Dockerized infra services, exactly as the existing `make up` / `make run` workflow already does. Don't force everyone into full-container rebuilds for every code change during development — that's for the "does the whole thing actually run" check and for deployment, not for iteration speed.

7. Backend contract (build against this exactly)

Base URL: configurable via environment variable, not hardcoded.

7.1 Health

`GET /v1/health` → `{"status": "ok", "version": "..."}"`

7.2 Ask (non-streaming)

`POST /v1/ask`

```
json
{
  "question": "string",
  "collection": "string",
  "mode": "auto | single | agentic",
  "filters": { "source": null },
  "options": { "top_m": 6, "return_chunks": true }
}
```

Response (answered):

json


```
{
  "run_id": "uuid", "trace_id": "string", "status": "answered",
  "answer": "string",
  "citations": [{ "index": 1, "chunk_id": "uuid", "source": "string", "section":
  "confidence": { "composite": 0.0, "retrieval": 0.0, "citation_coverage": 0.0,
  "mode_used": "string", "steps": 0, "degraded": [], "verified": true,
  "usage": { "latency_ms": 0, "tokens_in": 0, "tokens_out": 0, "cost_usd": 0.0
}
```

Response (abstained):

```
json
{
  "run_id": "uuid", "trace_id": "string", "status": "abstained",
  "abstention": { "found": ["string"], "missing": "string", "suggested_document
  "confidence": { "composite": 0.0, "retrieval": 0.0 }
}
```

7.3 Ask (streaming)

(POST /v1/ask/stream) — same request body, Server-Sent Events: (run_started), (step_started), (step_completed), (chunks_retrieved), (token), (verification), (answer_final), (error).

7.4 Documents

(GET /v1/documents) → list of indexed documents.

7.5 Errors

Every error response: ({ "error": { "code": "TT-XXXX", "message": "string", "trace_id": "string" } }). Surface (code) and (trace_id) in the UI on failure.

If any of these endpoints don't exist in the backend yet, stub the frontend call but mark the UI element "Coming soon" — do not simulate a response.

8. Decisions made, and the one still open

Resolved, build against these without asking again:

- Fonts: Inter (UI) + Fraunces (headline).
- Contact: srujeethvisinigiri@gmail.com, +91 91368 92409.
- Auth: Argon2id password hashing, short-lived access token + rotating refresh token, httpOnly/Secure/SameSite cookies, login rate limiting (Section 5) — build it, it doesn't exist yet.

Still open, mine to resolve later, not Antigravity's to guess:

☐ Replace the temporary CloudFront background video with one I actually own the rights to, before this is shown publicly.

9. Read the project documentation first

Before writing any code, read everything under `(docs/)` in this repository (the master engineering documentation and the work breakdown structure). It is the source of truth for the data model, the API contract, the architecture and the retrieval/agent design. Where this brief and `(docs/)` overlap, `(docs/)` wins on backend behavior; this brief wins on frontend design and copy. Where something isn't covered by either, ask rather than invent.

One gap to fill while reading: the master documentation's database schema (Section 6.3) doesn't yet include `(users)` or `(refresh_tokens)` tables, because auth wasn't designed when it was written. Add them, following the same conventions as the existing schema (UUID primary keys, `(created_at)`/`(updated_at)`, Alembic migration).